

Nama Lengkap : MUHAMAD FATIR KURNIAWAN
NIM : J0403251057
Kelas : TPL A1

1. Penjelasan Node dan Edge

Node adalah titik atau simpul dalam graf yang merepresentasikan suatu entitas. Sementara itu, edge adalah garis atau sisi yang menghubungkan dua node dan merepresentasikan hubungan langsung di antara keduanya.

2. Adjacency List vs Adjacency Matrix

Adjacency List menyimpan graf sebagai kumpulan daftar, di mana setiap node hanya mencatat tetangga langsungnya. Sebaliknya, Adjacency Matrix menyimpan seluruh kemungkinan pasangan node dalam tabel $V \times V$ ($6 \times 6 = 36$ sel), di mana nilai 1 menandakan adanya edge dan 0 berarti tidak ada; akibatnya, 24 dari 36 sel terisi 0 dan ruang terbuang sia-sia. Dari perbandingan ini, Adjacency List jauh lebih efisien dari segi memori ($O(V+E)$ vs $O(V^2)$) dan lebih cepat dalam traversal berbasis tetangga seperti BFS/DFS.

3. Deskripsi Graf

Graf ini merepresentasikan konektivitas jalan antara 6 kota di Jawa Timur. Setiap node mewakili sebuah kota, dan setiap edge merepresentasikan jalan langsung yang menghubungkan dua kota tanpa melewati kota lain.

Node (Kota):

- Surabaya (Hub utama, 2 koneksi)
- Malang (2 koneksi)
- Mojokerto (1 koneksi (leaf node))
- Sidoarjo (2 koneksi)
- Pasuruan (Hub utama, 3 koneksi)
- Probolinggo (2 koneksi)

Edge (Jalan):

- Surabaya — Sidoarjo
- Surabaya — Mojokerto
- Sidoarjo — Pasuruan

- Pasuruan — Malang
- Pasuruan — Probolinggo
- Malang — Probolinggo

4. Jenis Graf: Undirected

Graf ini adalah undirected graph karena hubungan antar kota bersifat dua arah. Jika ada jalan dari Surabaya ke Sidoarjo, maka secara otomatis ada jalan balik dari Sidoarjo ke Surabaya. Tidak ada arah khusus pada setiap edge.

5. Alasan Pemilihan Edge

Edge dipilih berdasarkan konektivitas langsung antar kota, artinya dua kota dihubungkan hanya jika ada jalan langsung tanpa melewati kota lain.

6. Representasi yang Lebih Cocok

Graf ini lebih cocok menggunakan Adjacency List karena jumlah edge aktual jauh lebih sedikit dari edge maksimal yang mungkin ada.

7. Dense atau Sparse?

Graf ini termasuk **sparse graph**. Berikut perhitungan density-nya:

Rumus Density Graf:

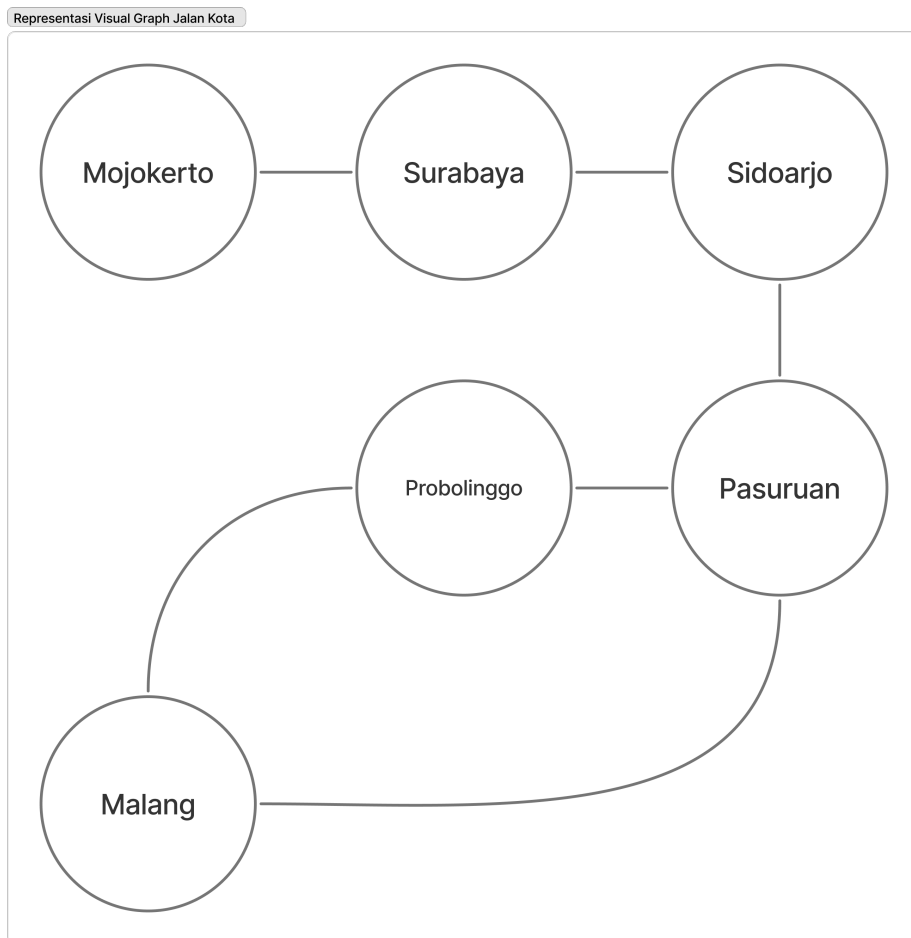
$$\begin{aligned}
 \text{Density} &= (2 \times E) / (V \times (V-1)) \\
 &= (2 \times 6) / (6 \times 5) \\
 &= 12 / 30 \\
 &= 0.4 \rightarrow 40\%
 \end{aligned}$$

Dengan density 40%, graf ini masuk kategori sparse, masih ada banyak pasangan kota yang belum terhubung langsung. Oleh karena itu, Adjacency List adalah pilihan representasi yang lebih efisien.

8. Kesimpulan Singkat

Graf konektivitas kota-kota Jawa Timur ini merupakan undirected graph dengan 6 node dan 6 edge yang bersifat sparse (density 40%). Representasi terbaik untuk graf ini adalah Adjacency List karena lebih hemat memori ($O(V+E)$) dibandingkan Adjacency Matrix ($O(V^2)$), dan lebih efisien dalam operasi penelusuran seperti BFS maupun DFS.

9. Output dan Representasi Visual



```

Pertemuan11_MUHAMAD FATIR KURNIAWAN_J0403251057.py X
Praktikum11 > Pertemuan11_MUHAMAD FATIR KURNIAWAN_J0403251057.py > printAdjacencyList

70
71 def printAdjacencyList(graph):
72     print("Adjacency List:")
73     for node, neighbors in graph.items():
74         print(f" {node}: {neighbors}")
75
76
77 def printAdjacencyMatrix(nodes, matrix):
78     print("Adjacency Matrix:")
79     for i in range(len(matrix)):
80
81 tsukuru in algo-struktur-data-tp12106 on ? main [?]
> python3 Praktikum11/Pertemuan11_MUHAMAD\ FATIR\ KURNIAWAN_30403251057.py
at 14:40:56 38%
Node (Kota):
- Surabaya
- Malang
- Mojokerto
- Sidoarjo
- Pasuruan
- Probolinggo

Edge (Jalan):
Surabaya -- Sidoarjo
Surabaya -- Mojokerto
Sidoarjo -- Pasuruan
Pasuruan -- Malang
Malang -- Probolinggo

Adjacency List:
Surabaya: ['Sidoarjo', 'Mojokerto']
Malang: ['Pasuruan', 'Probolinggo']
Mojokerto: ['Surabaya']
Sidoarjo: ['Surabaya', 'Pasuruan']
Pasuruan: ['Sidoarjo', 'Malang', 'Probolinggo']
Probolinggo: ['Pasuruan', 'Malang']

Adjacency Matrix:
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
0 1 0 1
0 1 0 1

81 tsukuru in algo-struktur-data-tp12106 on ? main [?]
>
at 14:40:57 38%

```

Output Program Praktikum 1—3

```

tsukuru in algo-struktur-data-tp12106 on ? main [?] took 5s
> python3 Praktikum11/Praktikum1.py
Adjacency Matrix:
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0

```

```

tsukuru in algo-struktur-data-tp12106 on ? main [*?]
> python3 Praktikum11/Praktikum2.py
Adjacency List:
A: ['B', 'C']    B    C
B: ['A', 'D']    A    D
C: ['A', 'D']    A    D
D: ['B', 'C']    B    C

```

```

tsukuru in algo-struktur-data-tp12106 on ? main [*?]
> python3 Praktikum11/Praktikum3.py
Adjacency List:
0: [1, 2]
1: [0, 2]
2: [0, 1, 3]
3: [2]

```